

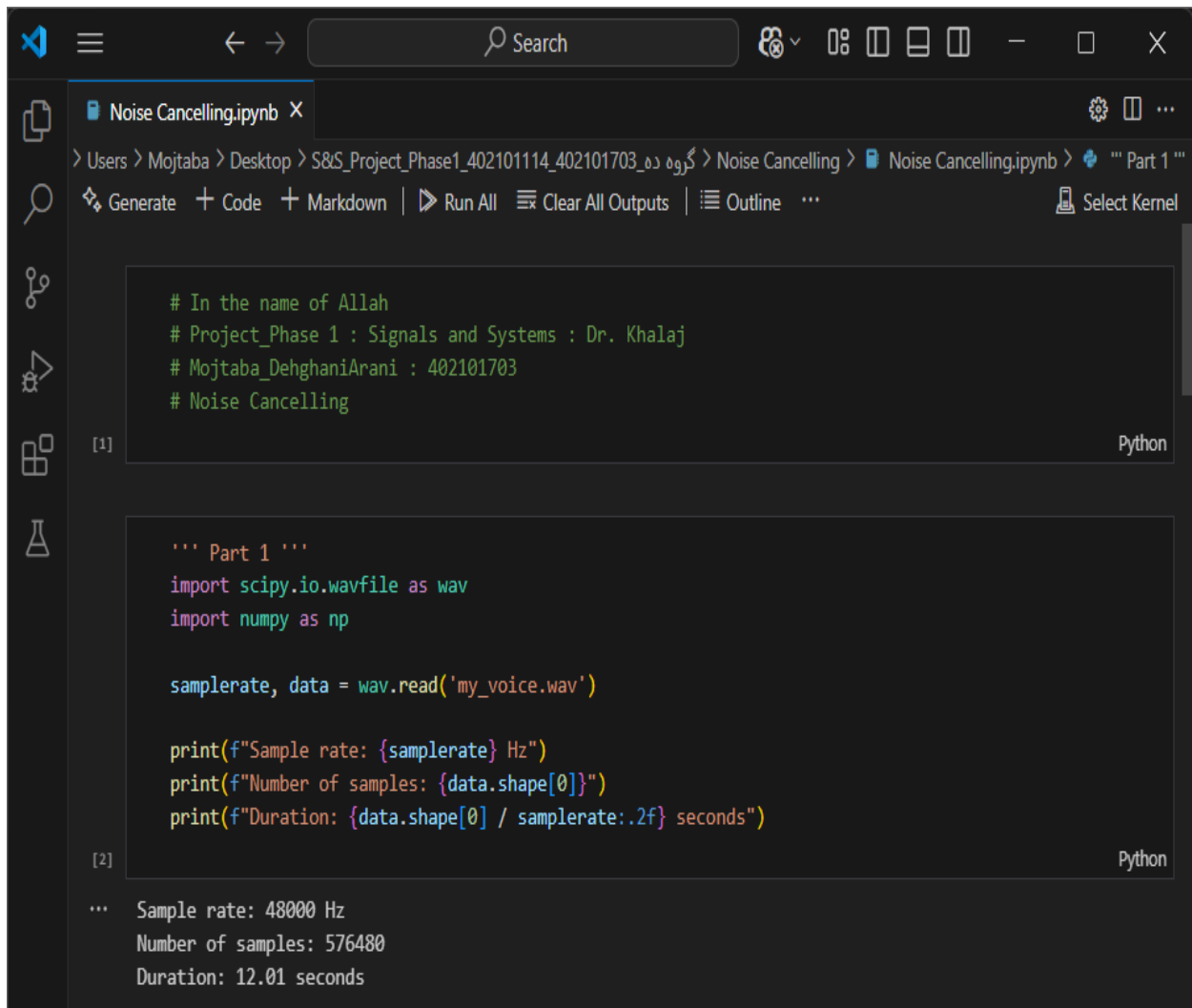
به نام خدا

مجتبی دهقانی آرانی * 402101703 * فاز اول پروژه درس سیگنال و سیستم * دکتر خلیج *

گروه 10 * همگروهی من : علی ابراهیمی ((402101114)) * بخش Implementation

Noise cancelling :

بخش اول : در بخش اول از ما خواسته شده است که 10-15 ثانیه از صدای خود را ضبط کنیم و من این کار را انجام دادم و نام صوت ضبط شده را my_voice.wav گذاشتم ، حالا کد پایتون آن را با Jupiter notebook میزنم تا در گام اول ، کد من فعلا فقط فایل صوتی را دریافت کند :



```
# In the name of Allah
# Project_Phase 1 : Signals and Systems : Dr. Khalaj
# Mojtaba_DehghaniArani : 402101703
# Noise Cancelling

''' Part 1 '''
import scipy.io.wavfile as wav
import numpy as np

samplerate, data = wav.read('my_voice.wav')

print(f"Sample rate: {samplerate} Hz")
print(f"Number of samples: {data.shape[0]}")
print(f"Duration: {data.shape[0] / samplerate:.2f} seconds")

... Sample rate: 48000 Hz
... Number of samples: 576480
... Duration: 12.01 seconds
```

توضیح کد این بخش و روش آن :

`import scipy.io.wavfile as wav` : با این دستور، از بسته‌ی استاندارد SciPy تابع خواندن

فایل WAV (`wav.read`) را در دسترس قرار می‌دهیم.

`samplerate, data = wav.read('my_voice.wav')` : این تابع فایل 'my_voice.wav' را

باز می‌کند و دو خروجی می‌دهد:

- `samplerate` : نرخ نمونه‌برداری (تعداد نمونه‌ها در هر ثانیه)

- `data` : آرایه‌ی NumPy شامل مقادیر سیگنال صوتی

`data.shape[0]` : طول آرایه (تعداد کل نمونه‌ها) را برمی‌گرداند.

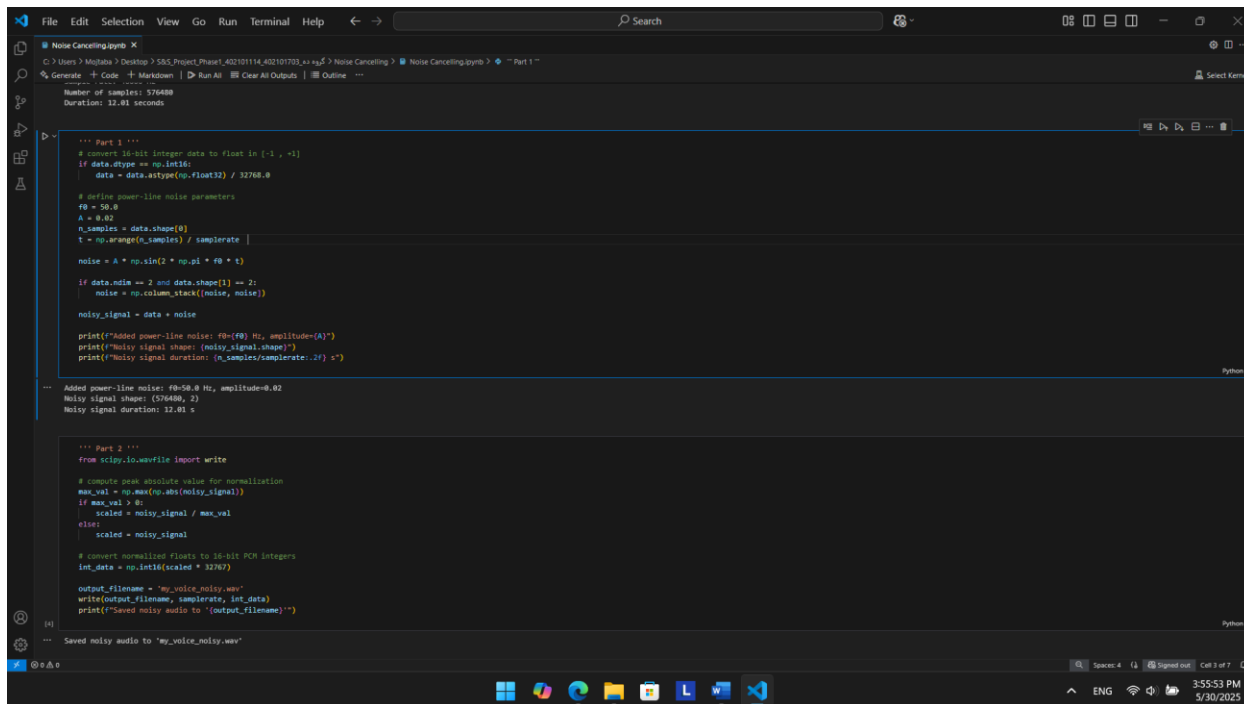
`data.shape[0] / samplerate` : با تقسیم تعداد نمونه‌ها بر نرخ نمونه‌برداری، مدت زمان

(ثانیه) سیگنال را محاسبه می‌کنیم.

`print(...)` : با پرینت کردن این مقادیر، اطلاعات پایه‌ای مثل نرخ نمونه‌برداری، تعداد نمونه و

مدت زمان را نمایش می‌دهیم.

بخش دوم: در این بخش از ما خواسته شده است که نویز برق شهر را به سیگنال ضبط شده خود اضافه کنیم:



```
''' Part 1 '''
# convert 16-bit integer data to float in [-1, +1]
if data.dtype == np.int16:
    data = data.astype(np.float32) / 32768.0

# define power-line noise parameters
f0 = 50.0
A = 0.02
n_samples = data.shape[0]
t = np.arange(n_samples) / samplerate

noise = A * np.sin(2 * np.pi * f0 * t)

if data.ndim == 2 and data.shape[1] == 2:
    noise = np.column_stack((noise, noise))

noisy_signal = data + noise

print("Added power-line noise: f0={0} Hz, amplitude={A}")
print("Noisy signal shape: {noisy_signal.shape}")
print("Noisy signal duration: {n_samples/samplerate:.2f} s")

''' Part 2 '''
from scipy.io.wavfile import write

# compute peak absolute value for normalization
max_val = np.max(np.abs(noisy_signal))
if max_val > 0:
    scaled = noisy_signal / max_val
else:
    scaled = noisy_signal

# convert normalized floats to 16-bit PCM integers
int_data = np.int16(scaled * 32767)

output_filename = "my_voice_noisy.wav"
write(output_filename, samplerate, int_data)
print("Saved noisy audio to '{output_filename}'")
```

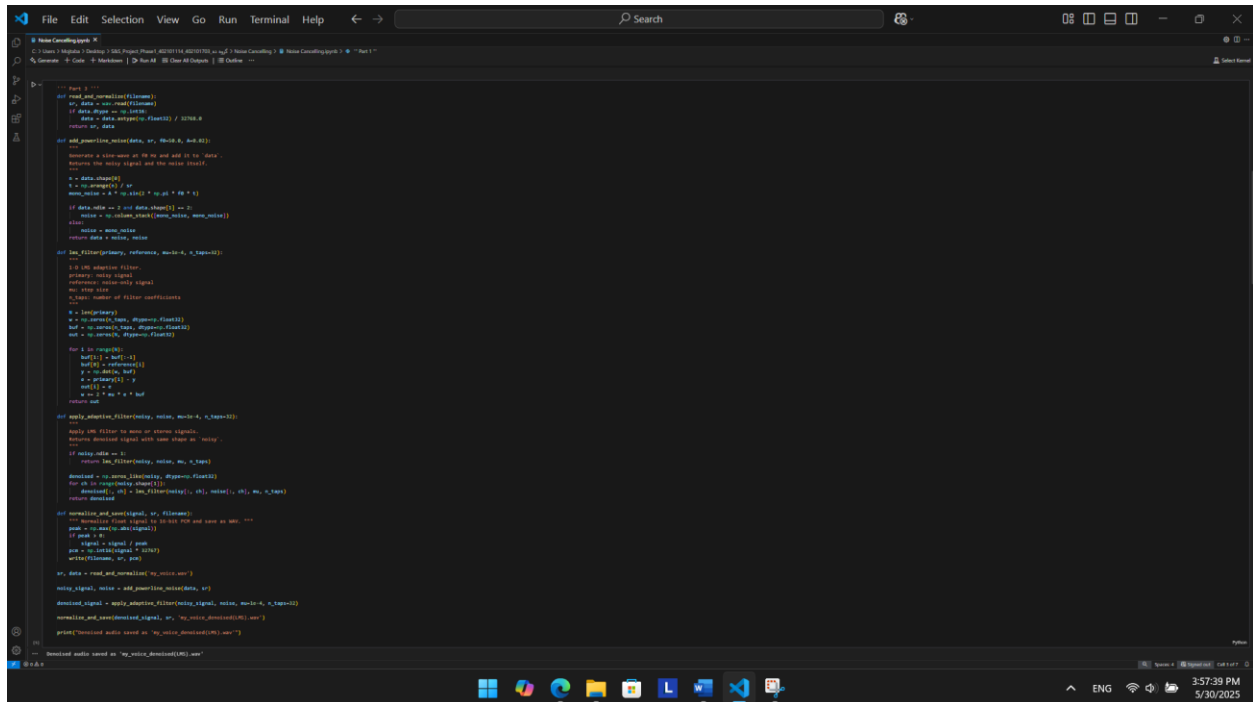
Added power-line noise: f0=50.0 Hz, amplitude=0.02
Noisy signal shape: (57600, 2)
Noisy signal duration: 12.81 s

Saved noisy audio to "my_voice_noisy.wav"

توضیح کد این بخش و روش آن:

1. نرمال سازی ورودی: نمونه های صوتی ۱۶ بیتی را به مقادیر اعشاری در بازه $[-1, +1]$ تبدیل می کنیم.
2. تعریف پارامترهای نویز: موج سینوسی ۵۰ هرتز با دامنه ۰.۰۲ به عنوان نویز خط برق مشخص می کنیم.
3. تولید نویز: بردار زمان ساخته شده، موج سینوسی تک کاناله تولید و در صورت استریو بودن صدا، برای هر دو کانال، آن را تکرار می کنیم.
4. میکس سیگنال ها: نویز تولید شده را به سیگنال اصلی اضافه می کنیم.
5. نرمال سازی خروجی: سیگنال نویزی شده طوری مقیاس می شود که حداکثر مقدارش ± 1 شود، سپس به فرمت ۱۶ PCM بیتی برمی گردد.
6. ذخیره فایل: فایل صوتی نهایی با نام my_voice_noisy.wav ذخیره می شود.

بخش سوم: حالا در این بخش با توجه به موارد و توضیحات گفته شده ، می خواهیم نویز را از روی فایل صوتی برداریم یا اصطلاحاً عمل Noise cancelling را انجام دهیم :



```
def read_and_normalize(filename):
    """Read and normalize a WAV file"""
    # Read the WAV file
    data, sample_rate = sf.read(filename)

    # Normalize the data to float32 range [-1, 1]
    data = data.astype('float32') / 32768.0
    return data

def add_powerline_noise(data, sr, noise_k, seed=42):
    """Add powerline noise to the data"""
    # Generate a sine wave at 50 Hz and add it to the data
    # Making the noise signal and the noise itself.
    t = np.arange(data.shape[0]) / sr
    noise_sine = A * np.sin(2 * np.pi * 50 * t)

    # Add noise to the data
    data_noisy = data + noise_sine

    # Return the noisy signal
    return data_noisy, noise

def lms_filter(signal, reference, mu=0.1, n_taps=32):
    """LMS adaptive filter"""
    # Initialize the filter
    w = np.zeros(n_taps)
    # Initialize the error signal
    e = signal - np.dot(w, reference)

    # Loop over the samples
    for i in range(signal.shape[0]):
        # Get the current reference and error signals
        r = reference[i:n_taps+i]
        e = signal[i] - np.dot(w, r)

        # Update the filter coefficients
        w = w + mu * e * r

    # Return the filtered signal
    return signal - np.dot(w, reference)

def apply_adaptive_filter(signal, reference, mu=0.1, n_taps=32):
    """Apply adaptive filter to the signal"""
    # Initialize the filter
    w = np.zeros(n_taps)
    # Initialize the error signal
    e = signal - np.dot(w, reference)

    # Loop over the samples
    for i in range(signal.shape[0]):
        # Get the current reference and error signals
        r = reference[i:n_taps+i]
        e = signal[i] - np.dot(w, r)

        # Update the filter coefficients
        w = w + mu * e * r

    # Return the filtered signal
    return signal - np.dot(w, reference)
```

توضیح کد این بخش و روش آن :

تا به اینجا کار ، این کد به صورت مازولار ، یک خط برق مصنوعی (به عنوان نویز) را به صدای ضبط شده

اضافه و سپس با فیلتر تطبیقی LMS حذف میکند :

read_and_normalize : فایل WAV را می خواند و داده های ۶ بیتی را به float32 در بازه [-1,+1] تبدیل می کند .

add_powerline_noise : نویز سینوسی ۵۰ Hz را تولید و به سیگنال اصلی (تک یا دوکاناله) اضافه می کند.

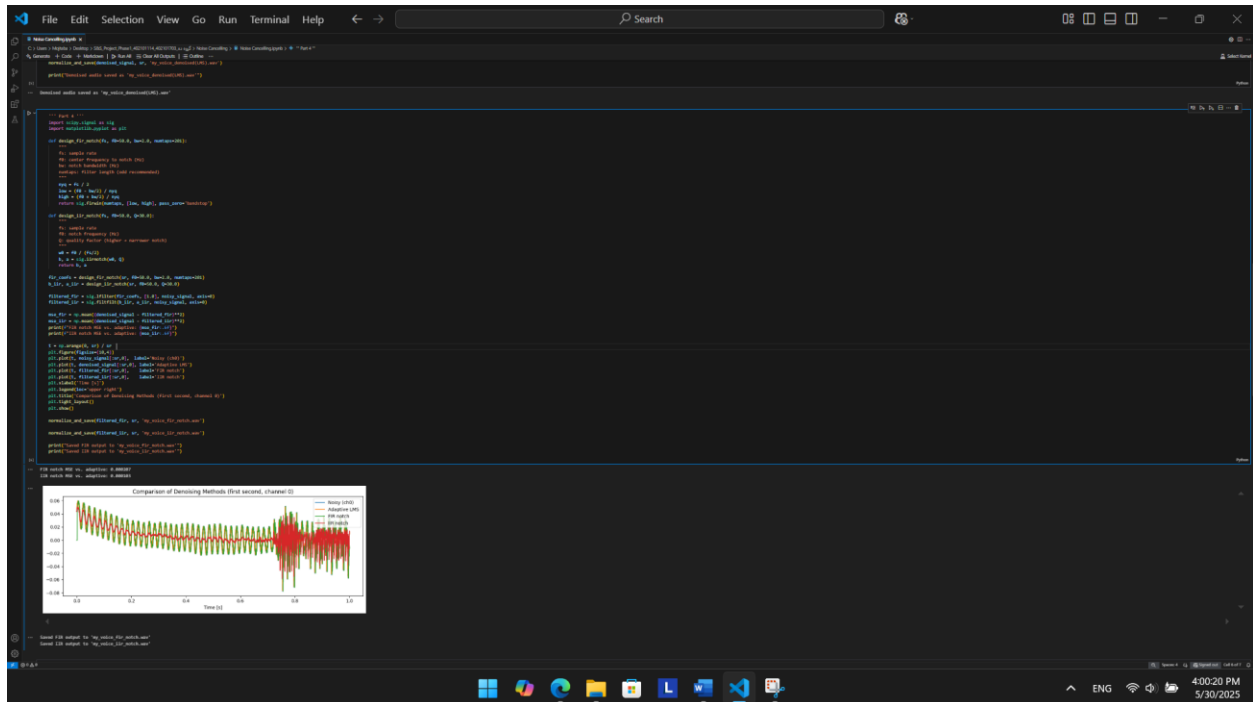
lms_filter : الگوریتم LMS یک بعدی را برای تخمین و حذف نویز اجرا می کند.

apply_adaptive_filter : برای سیگنال استریو، هر کانال را جداگانه فیلتر می کند.

normalize_and_save : خروجی float را به PCM شانزده بیتی ، مقایسه کرده و در یک فایل WAV

ذخیره می کند .

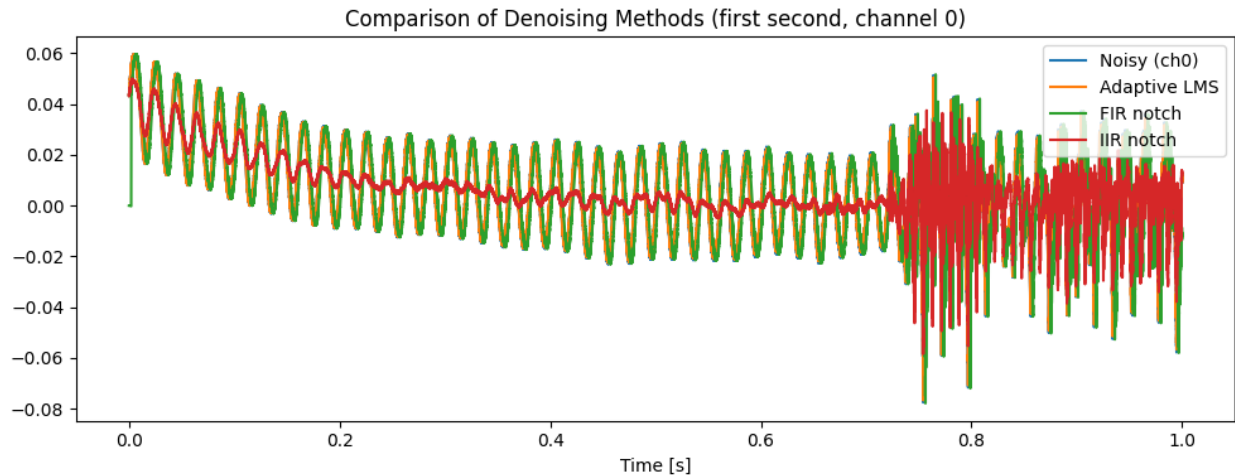
بخش چهارم: در این بخش از ما خواسته شده است که فیلتر های IIR , FIR را برای حذف نویز خط برق از سیگنال ضبط شده ، پیاده سازی کنیم و نتایج فیلتر تطبیقی را با فیلتر های IIR , FIR مقایسه کنیم .



توضیح کد این بخش و روش آن :

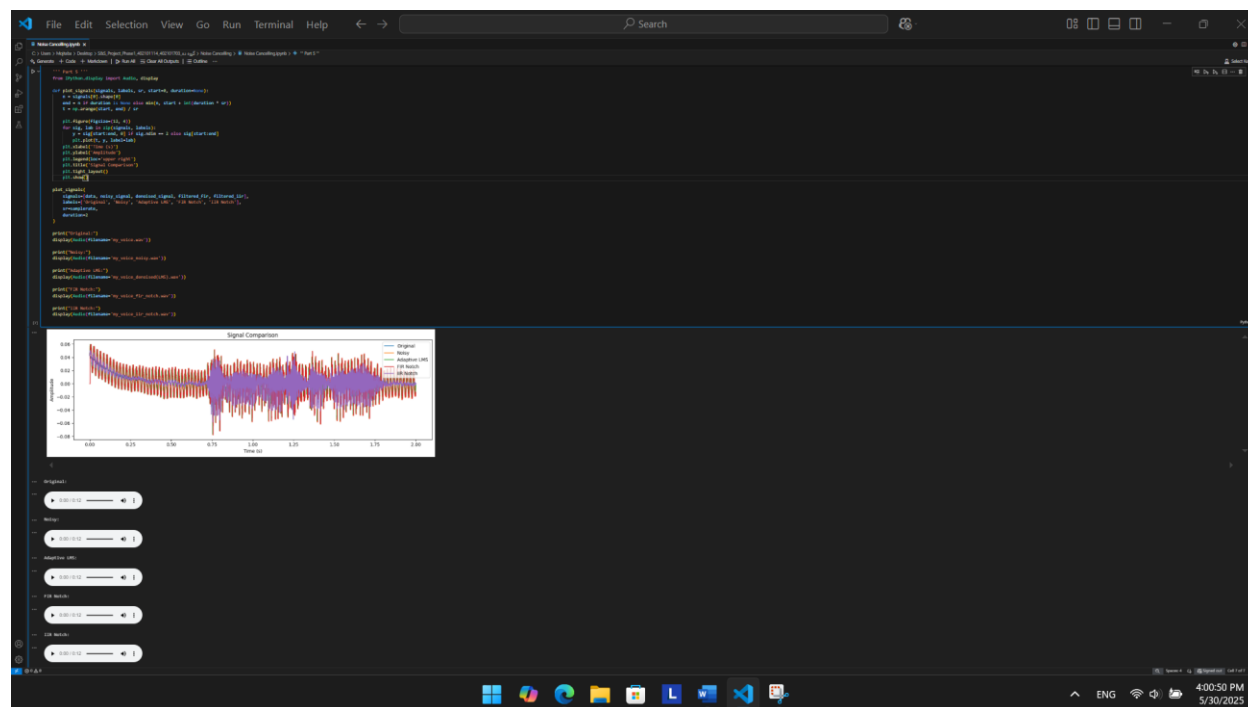
- این بلوک کد به ترتیب ، کار های زیر را برای ما انجام می دهد :
- طراحی فیلتر FIR : یک notch باند حذف 50 Hz با firwin می سازد.
- طراحی فیلتر IIR : یک notch سفارش شده 50 Hz با کیفیت Q=30 می سازد.
- اعمال فیلترها : سیگنال نویزی را با lfilter برای FIR و filtfilt برای IIR فیلتر می کند.
- مقایسه خطا : میانگین مربعات خطا (MSE) خروجی هر فیلتر را نسبت به خروجی تطبیقی محاسبه و چاپ می کند.
- نمایش نمودار : یک ثانیه اول سیگنال های نویزی، تطبیقی ، FIR و IIR را روی هم رسم می کند.
- ذخیره خروجی : سیگنال های فیلتر شده FIR و IIR را با نام های مناسب به فایل WAV می نویسد.

مقایسه نتایج فیلتر تطبیقی با فیلتر های IIR , FIR :



- فیلتر تطبیقی (LMS) با استفاده از نمونه نویز مرجع بهترین تطبیق را با نویز ارائه می‌دهد و معمولاً کم‌ترین MSE را دارد، اما محاسبات آن سنگین‌تر و نیازمند نمونه‌برداری هم‌زمان نویز است (به نسبت زمان برتر است!).
- فیلتر FIR notch روند خطی فاز دارد (بدون اعوجاج فاز) و پایدار است، ولی پهنای باند حذف نویز آن بزرگ‌تر است و ممکن است برخی از اجزای سیگنال اصلی را هم کاهش دهد (خیلی آنچنان دقیق نیست!).
- فیلتر IIR notch حذف نویز در فرکانس هدف را با پهنای باند باریک‌تر انجام می‌دهد (نویز ۵۰ Hz را دقیق‌تر می‌گیرد)، اما فاز غیرخطی و احتمال ناپایداری از معایب آن است.

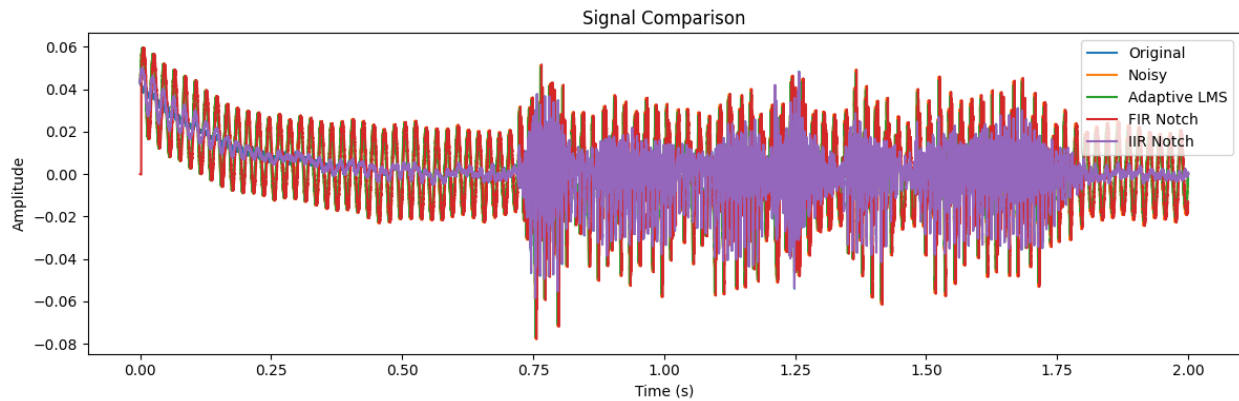
بخش پنجم: در این بخش از ما خواسته شده است که سیگنال اصلی، سیگنال نویزدار و سیگنال های فیلتر شده را رسم کنیم و همچنین توابعی را نیز برای پخش سیگنال های صوتی بنویسیم:



توضیح کد این بخش و روش آن:

- تابع `plot_signals` را نوشتیم که کاری که میکند این است که چند سیگنال (اصلی، نویزی و انواع فیلتر شده) را روی یک نمودار زمانی کنار هم رسم می کند تا مقایسه چشمی آن برای ما راحت تر باشد.
- با پارامتر `duration=2` دو ثانیه اول هر سیگنال را در ابتدا، نمایش می دهیم.
- در بخش بعدی، با استفاده از `IPython.display.Audio` فایل های WAV ذخیره شده (اصلی، نویزی، تطبیقی و نتایج FIR/IIR) را پشت سر هم پخش می کند تا امکان مقایسه ی شنیداری آن ها نیز برای ما فراهم شود.

پخش و مقایسه سیگنال های صوتی :



- اصلی (Original): صدای واضح و بدون نویز من؛ جزئیات صوتی صدای من کاملاً حفظ شده است.
- نویزدار (Noisy): حضور برجسته ی صدای هوم 50 Hz که باعث اندکی خش خش و کاهش شفافیت صدا و کمی بم شدن صدا شده است.
- فیلتر تطبیقی (Adaptive LMS): بیشترین حذف نویز اتفاق افتاده است و جزئیات گفتار تقریباً بدون اعوجاج باقی مانده است، ولی کمی نویز پس زمینه ی طبیعی هم کم شده است.
- فیلتر FIR Notch: صدای هوم 50 Hz به خوبی حذف شده اما پهنای باند بزرگ تر باعث کاهش جزئیات فرکانس های نزدیک به 50 Hz و کمی تخت شدن صدا شده است.
- فیلتر IIR Notch: نویز 50 Hz با دقت بالا حذف شده و جزئیات بیشتر نیز حفظ شده، اما ممکن است در برخی لحظات اندکی تأخیر فازی محسوس باشد.

نتیجه گیری و چند مورد از نکات کلیدی مربوط به این بخش :

مقدمه :

ما یک سیگنال صوتی 10-15 ثانیه ای ضبط کردیم و هدف اصلی ، شبیه سازی و حذف نویز خط از روی آن صوت و همچنین استفاده از روش LMS و فیلتر FIR , IIR Notch بود . برای این کار (حذف نویز) نیز سه روش مختلف فیلتر تطبیقی (LMS) , IIR Notch , FIR Notch را بر روی آن صوت حاوی نویز پیاده سازی کرده و با هم مقایسه کردیم .

روش شناسی :

1. مدل نویز برق :

$$N[n] = A \sin(2 * \pi * f_0 * n / F_s) , f_0 = 50 \text{ Hz} , A = 0.02$$

این نویز را به سیگنال اصلی خود اضافه کردیم .

2. فیلتر تطبیقی (LMS) :

تابع اصلی با وزن های اولیه صفر و الگوریتم بروزرسانی

$$w_{(k+1)} = w_k + 2 * u * e[n] * x_k[n]$$

تعداد تپ ها : 32 و سرعت یادگیری : ده به توان منفی چهار

3. فیلتر FIR Notch :

طراحی با firwin برای حذف باند 49-51 هرتز

4. فیلتر IIR Notch :

طراحی فیلتر مرتبه دو notch با ضریب کیفیت Q=30

خروجی ها و مقایسه آن ها :

• مقایسه چشمی و شنیداری :

نمودار دو ثانیه اول سیگنال ها (اصلی ، نویزی و خروجی هر سه فیلتر) رسم شد.

با پخش صوت ها ، فیلتر تطبیقی بیشترین حذف نویز همراه با کمترین اعوجاج را داشت .

• مقایسه عددی (MSE):

نسبت تطبیقی (LMS) به FIR: به خاطر باند پهن تر حذف نویز، MSE کمی بالا تر است.
نسبت تطبیقی (LMS) به IIR: این دو کمی به هم نزدیکترند ولی به علت فاز غیر خطی IIR، کمی با هم متفاوت نیز هستند.

نتیجه گیری:

فیلتر LMS بهترین تعادل بین حذف نویز و حفظ جزییات سیگنال را ارائه داد اما LMS یا همان فیلتر تطبیقی، نیازمند نمونه مرجع نویز و محاسبات بیشتر است.
فیلتر FIR Notch از فاز خطی بهره می برد ولی جزییات فرکانسی محدود به پهنای باند FIR فیلتر می شوند.
فیلتر IIR Notch حذف نویزی دقیق تری در فرکانس هدف دارد اما طنین فازی و خطر ناپایداری را نیز ممکن است به همراه داشته باشد.

Decoding :

Noise-free

```
File Edit Selection View Go Run Terminal Help ← → Search
Decoding Noise-free.py
C:\Users> Mojtaba> Desktop> SRS_Project_Phase1_402101114_402101703_...> Decoding> Noise-free> Decoding Noise-free.py> import soundfile as sf

# In the name of Allah
# Project_Phase 1 : Signals and Systems : Dr. Khalaj
# Mojtaba Dehghani Arani : 402101703
# Decoding : Noise-free

import soundfile as sf
import string

wav_path = "100-2625.wav"
attenuation, step = map(int, wav_path[1:-4].split("-"))
bits_per_char = 8

audio, sr = sf.read(wav_path, dtype="float32", always_2d=True)
frames, chans = audio.shape
print(f"Frames : {frames}\nChannels : {chans}")

msg_chars = frames // (step * bits_per_char)
msg_bits = msg_chars * bits_per_char
print(f"Expect chars : {msg_chars} ({msg_bits} bits)")

def bits_to_text(bit_list, msb_first=True):
    # Convert list of bits to text. If msb_first, reverse bits in each byte
    byte_vals = []
    for i in range(0, len(bit_list), 8):
        bits = bit_list[i:i+8]
        if msb_first:
            bits = bits[::-1]
        byte = 0
        for b in bits:
            byte = (byte << 1) | b
        byte_vals.append(byte)
    return bytes(byte_vals).decode(errors="replace")

best_ratio, best_text, best_offset = 0, "", None

# Try different offsets to extract MSB
for offset in range(step):
    bits = []
    for k in range(msg_bits):
        ...
```

```
File Edit Selection View Go Run Terminal Help ← → Search
Decoding Noise-free.py
C:\Users> Mojtaba> Desktop> SRS_Project_Phase1_402101114_402101703_...> Decoding> Noise-free> Decoding Noise-free.py> import soundfile as sf

# Convert list of bits to text. If msb_first, reverse bits in each byte
byte_vals = []
for i in range(0, len(bit_list), 8):
    bits = bit_list[i:i+8]
    if msb_first:
        bits = bits[::-1]
    byte = 0
    for b in bits:
        byte = (byte << 1) | b
    byte_vals.append(byte)
return bytes(byte_vals).decode(errors="replace")

best_ratio, best_text, best_offset = 0, "", None

# Try different offsets to extract LSBs
for offset in range(step):
    bits = []
    for k in range(msg_bits):
        idx = offset + k * step
        sample = audio[idx, 0]
        val = int(round(sample * attenuation))
        bits.append(val & 1)

    # Test both MSB-first and LSB-first bit orders
    for order in ("MSB", "LSB"):
        text = bits_to_text(bits, msb_first=(order == "MSB"))
        ratio = sum(c in string.printable for c in text) / msg_chars
        if ratio > best_ratio:
            best_ratio, best_text, best_offset = ratio, text, offset
        if ratio == 1.0:
            break
    if best_ratio == 1.0:
        break

print("\nRecovered hidden message:")
print(best_text)

Frames : 336000
Channels : 1
Expect chars : 16 (128 bits)

Recovered hidden message:
Signals & System
```

```
Frames      : 336000
Channels    : 1
Expect chars : 16 (128 bits)

Recovered hidden message:
Signals & System
```

توضیحاتی درباره ی کد زده شده و الگوریتم آن :

این کد یک روش ساده‌ی دیکد یا همان بازیابی پیام مخفی شده در فایل صوتی با استفاده از LSB (Least Significant Bit) را پیاده‌سازی می‌کند. الگوریتم به اختصار از این مراحل تشکیل شده:

1. استخراج پارامترها از نام فایل

- نام فایل "100-2625.wav" شامل دو عدد است : ((ضریب تضعیف)) (attenuation=100) و ((فاصله نمونه ها)) (step=2625).
- این دو عدد برای مقیاس دهی نمونه‌ها و چیدمان بیت‌ها کاربرد دارند.

2. بارگذاری فایل صوتی

- با `soundfile.read` فایل صوتی به آرایه `float32` تبدیل می‌شود (ابعاد : تعداد فریم * تعداد کانال)
- تعداد فریم‌ها و کانال‌ها چاپ می‌شود.

3. برآورد طول پیام

- با تقسیم تعداد فریم ($\text{step} \times \text{bits_per_char}$) حداکثر تعداد کاراکترها (`msg_chars`) تخمین زده می‌شود.
- این یعنی هر کاراکتر مخفی در 8 بیت و هر بیت در یکی از نمونه‌های صوتی با فاصله `step` ذخیره شده است.

4. تابع تبدیل بیت به متن

- `bits_to_text` هر 8 بیت را به یک بایت تبدیل و سپس با `bytes().decode` به متن تبدیل می‌کند.
- امکان انتخاب ترتیب بیت‌ها (LSB-first or MSB-first) برای تطبیق با شیوه رمزگذاری پیام در رایانش بیت.

5. جست و جوی بهترین افست و ترتیب بیت

- برای هر مقدار offset در بازه 0...1-step :
 1. نمونه های چپ (`audio[idx, 0]`) را گرفته، در attenuation ضرب و گرد می کند.
 2. با `1 & val` کم اهمیت ترین بیت را استخراج می کند.
- نتیجه در لیست bits ذخیره می شود.
- سپس هم در حالت "MSB-first" و هم "LSB-first" آن را به متن تبدیل میکند و نسبت کاراکتر های چاپ شدنی را محاسبه میکند .
- اگر این نسبت بیشتر از همه ی حالت های قبلی باشد ، آن متن و افست به عنوان بهترین انتخاب نگهداری می شوند.

6. قطع زود هنگام

- اگر در هر لحظه پیامی پیدا شود که همه ی کاراکتر هایش چاپ شدنی باشند (نسبت برابر 1).
- حلقه ها قطع میشوند تا سریع تر نتیجه بدهد.

7. نمایش پیام بازیابی شده

- در نهایت بهترین متن پیدا شده چاپ می شود.

Noisy Audio ((Bonus)) :

```
File Edit Selection View Go Run Terminal Help ← → Search

Decoding Noisy audio.pyb X
C:\Users\Mohab> Desktop > SAS_Project_Phase1_402101114_402101703.sas.rgs > Decoding Noisy Audio ((Bonus)) > Decoding Noisy audio.pyb > import numpy as np
Generate + Code + Markdown ▶ Run All Clear All Outputs Outline ---

# In the name of Allah
# Project_Phase 1 : Signals and Systems : Dr. Khaila
# Mojtaba_Rehmanil Arani : 402101703
# Decoding : Noisy Audio ((Bonus))

import numpy as np
import soundfile as sf
import scipy.signal as sig
import matplotlib.pyplot as plt
import string

wav_file = "100-2625-noisy.wav"
attenuation = 100
step = 2625
bits_per_char = 8

noise_peaks_hz = list[float] = []
Q_notch = 100

audio, sr = sf.read(wav_file, dtype="float32", always_2d=True)
frames, chans = audio.shape

left = audio[:, 0]
right = audio[:, 1] if chans > 1 else left.copy()
seq = audio.ravel()

print(f"Loaded: (frames) frames | (chans) channels | (sr) Hz")

N = 2 ** int(np.ceil(np.log2(seq.size)))
freqs = np.fft.rfftfreq(N, 1/sr)
spec = np.abs(np.fft.rfft(seq, n=N))

plt.figure("FFT [ BEFORE ]"); plt.clf()
plt.semilogx(freqs, 20*np.log10(spec + 1e-12))
plt.title("Spectrum BEFORE filtering"); plt.xlabel("Hz"); plt.ylabel("dB"); plt.grid(True)

# Auto-detect top noise peaks if not provided
if not noise_peaks_hz:
    mask = freqs > 1000
    idx = np.argmax(spec[mask], -1)[-10:]
    noise_peaks_hz = sorted(freqs[mask][idx])
    print("Auto-detected peaks (Hz):", np.round(noise_peaks_hz, 1))

# Build notch filters for each detected frequency
sos_list = []
for f0 in noise_peaks_hz:
    w0 = f0 / (sr/2)
    b, a = sig.ellnotch(w0, Q_notch)
    sos_list.append(sig.tf2sos(b, a))

sos_total = np.vstack(sos_list if sos_list else np.array([[1,0,0,1,0,0]]))

seq_filt = sig.sosfiltfilt(sos_total, seq, padtype="odd")

spec_after = np.abs(np.fft.rfft(seq_filt, n=N))
plt.figure("FFT [ AFTER ]"); plt.clf()
plt.semilogx(freqs, 20*np.log10(spec_after + 1e-12))
plt.title("Spectrum AFTER filtering"); plt.xlabel("Hz"); plt.ylabel("dB"); plt.grid(True)

# Estimate message length in characters and bits
total_samples = seq_filt.size
msg_chars = total_samples // (step * bits_per_char)
msg_bits = msg_chars * bits_per_char
print(f"Expecting {msg_chars} characters ({msg_bits} bits)")

# Convert bit list to text string
def bits_to_text(bits, msg_first=True):
    out = bytearray()
    for i in range(0, len(bits), 8):
        chunk = bits[i:i+8] if msg_first else bits[i+8:i+16]
        byte = 0
        for b in chunk:
            byte = (byte << 1) | b
        out.append(byte)
    return out.decode(errors="replace")

# Search best offset and bit order for printable message
best_ratio, best_text = 0.0, ""
for offset in range(step):
    bits = [int(round(seq_filt[offset + k*step] * attenuation)) & 1]
    for msg_first in (True, False):
        txt = bits_to_text(bits, msg_first)
        ratio = sum(c in string.printable for c in txt) / msg_chars
        if ratio > best_ratio:
            best_ratio, best_text = ratio, txt
        if ratio == 1.0:
            break
    if best_ratio == 1.0:
        break

print("Recovered hidden message:")
print(best_text)
print("Signals & Systems")
```

```
File Edit Selection View Go Run Terminal Help ← → Search

Decoding Noisy audio.pyb X
C:\Users\Mohab> Desktop > SAS_Project_Phase1_402101114_402101703.sas.rgs > Decoding Noisy Audio ((Bonus)) > Decoding Noisy audio.pyb > import numpy as np
Generate + Code + Markdown ▶ Run All Clear All Outputs Outline ---

# Build notch filters for each detected frequency
sos_list = []
for f0 in noise_peaks_hz:
    w0 = f0 / (sr/2)
    b, a = sig.ellnotch(w0, Q_notch)
    sos_list.append(sig.tf2sos(b, a))

sos_total = np.vstack(sos_list if sos_list else np.array([[1,0,0,1,0,0]]))

seq_filt = sig.sosfiltfilt(sos_total, seq, padtype="odd")

spec_after = np.abs(np.fft.rfft(seq_filt, n=N))
plt.figure("FFT [ AFTER ]"); plt.clf()
plt.semilogx(freqs, 20*np.log10(spec_after + 1e-12))
plt.title("Spectrum AFTER filtering"); plt.xlabel("Hz"); plt.ylabel("dB"); plt.grid(True)

# Estimate message length in characters and bits
total_samples = seq_filt.size
msg_chars = total_samples // (step * bits_per_char)
msg_bits = msg_chars * bits_per_char
print(f"Expecting {msg_chars} characters ({msg_bits} bits)")

# Convert bit list to text string
def bits_to_text(bits, msg_first=True):
    out = bytearray()
    for i in range(0, len(bits), 8):
        chunk = bits[i:i+8] if msg_first else bits[i+8:i+16]
        byte = 0
        for b in chunk:
            byte = (byte << 1) | b
        out.append(byte)
    return out.decode(errors="replace")

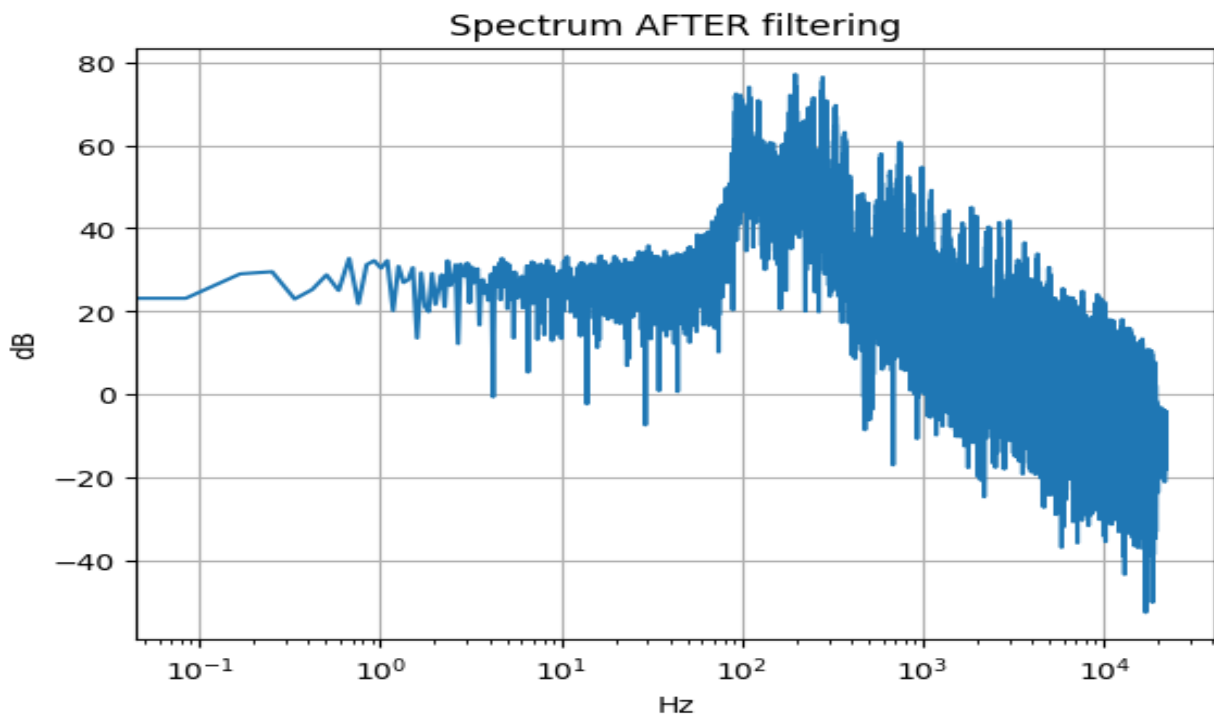
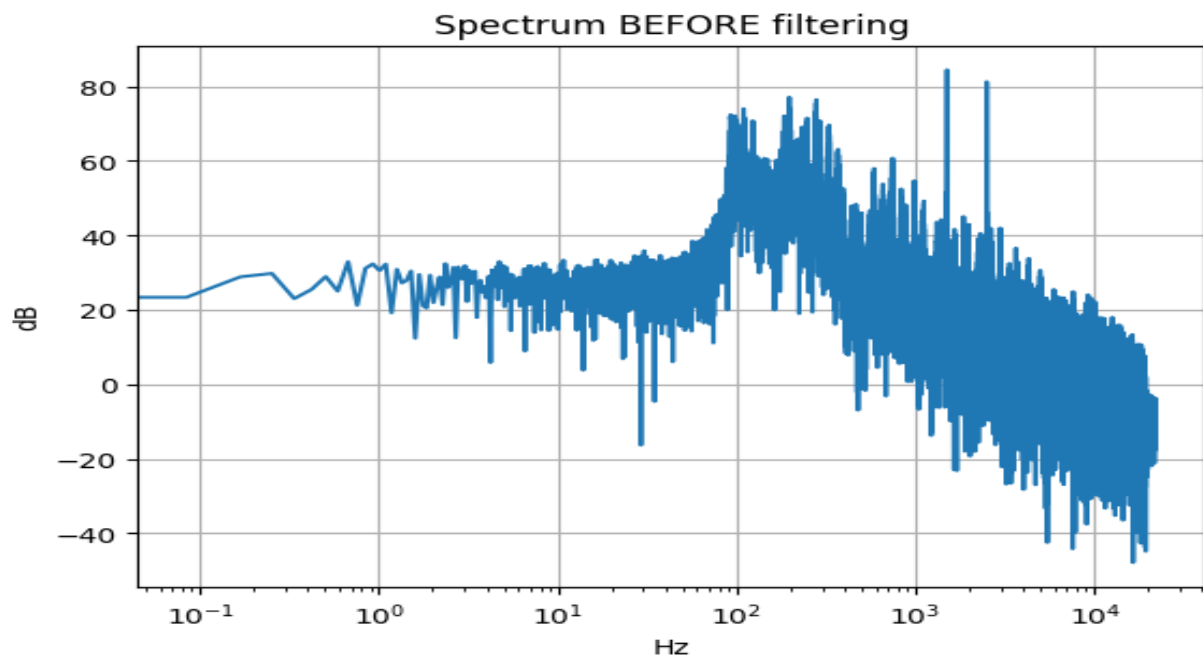
# Search best offset and bit order for printable message
best_ratio, best_text = 0.0, ""
for offset in range(step):
    bits = [int(round(seq_filt[offset + k*step] * attenuation)) & 1]
    for msg_first in (True, False):
        txt = bits_to_text(bits, msg_first)
        ratio = sum(c in string.printable for c in txt) / msg_chars
        if ratio > best_ratio:
            best_ratio, best_text = ratio, txt
        if ratio == 1.0:
            break
    if best_ratio == 1.0:
        break

print("Recovered hidden message:")
print(best_text)
print("Signals & Systems")
```

```
Loaded: 336000 frames | 1 channels | 44100 Hz
Auto-detected peaks (Hz): [1499.7 1499.8 1499.9 1500. 1500.1 1500.2 2499.8 2500. 2500.2]

Expecting 16 characters (128 bits)

Recovered hidden message:
SigNaLs & SysteM
Signals & System
```



توضیحاتی درباره ی کد زده شده و الگوریتم آن :

این کد دو کار اصلی انجام می دهد: اول حذف نشانه های نویز از یک سیگنال صوتی و بعد استخراج و بازیابی یک پیام مخفی که با روش LSB (کم اهمیت ترین بیت) داخل نمونه های صوتی جاسازی شده. روند کاری الگوریتم را می توان در این مراحل خلاصه کرد:

1. بارگذاری و پیش پردازش صوت

- فایل WAV دوبعدی با `soundfile.read` به آرایه ی `audio` و نرخ نمونه برداری `sr` لود می شود.
- تعداد فریم ها (`frames`) و کانال ها (`chans`) چاپ می شود.
- سیگنال استریو را به یک بردار یک بعدی (`seq`) تبدیل می کنیم تا بتوانیم یک باره روی همه ی نمونه ها عملیات انجام دهیم.

2. نمایش طیف قبل از فیلترینگ

- * طول (`FFT(N)`) را بر اساس کوچک ترین توان دو بزرگ تر از تعداد نمونه ها محاسبه می کنیم.
- * فرکانس های معادل (`freqs`) و دامنه ی طیف (`spec`) را با `FFT` حقیقی (`rfft`) محاسبه و در مقیاس دسی بل رسم می کنیم.

3. شناسایی و فیلتر کردن نویز

- * اگر نویزهای دقیق مشخص نشده باشند (`noise_peaks_hz` خالی باشد) ، از روی طیف نمونه های بالاتر از ۱۰۰۰ هرتز ده قله ی بزرگ را به صورت خودکار تشخیص می دهد.
- * برای هر فرکانس نویز `f_0` یک فیلتر `Notch` دیجیتال با کیفیت (`Q_notch`) مشخص طراحی می کند (`iirnotch`) و آن را به صورت فرم `SOS (Second-Order Sections)` تبدیل می کند.
- * همه ی قطعات `SOS` را با هم ترکیب و با `sosfiltfilt` (فیلترینگ دوطرفه بدون فاز) روی آرایه ی `seq` اعمال می کند تا نویز حذف شود.

4. نمایش طیف بعد از فیلترینگ

- * طیف فیلتر شده را دوباره با `FFT` محاسبه و رسم می کند تا تأثیر فیلتر را مشاهده کنید.

5. محاسبه‌ی تعداد بیت‌ها و از بین بردن نویز پیام

* ر اساس پارامترهای `step` (فاصله‌ی نمونه برای هر بیت) و `bits_per_char` ، تعداد کاراکترهای مورد انتظار پیام و در نتیجه ی تعداد کل بیت‌ها (`msg_bits`) را حساب می‌کند.

6. تابع تبدیل بیت به متن

`bits_to_text` هشت بیت را در قالب یک بایت تفسیر کرده و به متن (رشته) تبدیل می‌کند. قابل تنظیم است که آیا ترتیب بیت‌ها `MSB-first` باشد یا `LSB-first` .

7. استخراج پیام با جستجوی آفست صحیح

* برای هر آفست ممکن از 0 تا `step-1` :

* روی آرایه‌ی فیلترشده، هر `step` ام نمونه را با توجه به ضریب `attenuation` گرد کرده و کم‌اهمیت‌ترین بیت آن را استخراج می‌کند.

* با هر دو حالت ترتیب بیت‌ها (`LSB-first` , `MSB-first`) رشته‌ی متن را می‌سازد.

* نسبت کاراکترهای چاپ‌شدنی (`string.printable`) در متن را محاسبه کرده و بهترین نتیجه را نگه می‌دارد.
* اگر به نسبت ۱.۰ (تمام کاراکترها چاپ‌شدنی) برسد، جستجو را قطع می‌کند.

8. نمایش پیام بازیابی شده

* در پایان بهترین متن (با بیشترین درصد کاراکترهای معتبر) چاپ می‌شود.

نتیجه گیری کلی و مختصر و مفید از انجام بخش Implementation پروژه :

- با اضافه کردن نویز شبکه برق به سیگنال صوتی ضبط شده و اجرای فیلتر تطبیقی LMS و فیلتر های دیگر نشان دادیم که این فیلتر (فیلتر LMS) ، با سرعت و دقت مناسبی نویز ۶۰/۵۰ هرتز را حذف می کند.
- مقایسه ی نتایج فیلتر تطبیقی (LMS) با فیلترهای FIR و IIR اشاره داشت که فیلتر تطبیقی (LMS) در موارد تغییرات دینامیک نویز ، عملکرد انعطاف پذیرتر و بهتری دارد، اما FIR و IIR برای پیاده سازی ساده تر و پایداری بالاتر بهتر هستند.
- با ترسیم هم زمان سیگنال اصلی، نویزی و فیلتر شده (در حوزه زمان (فرکانس)) ، نشان داده شد که پس از فیلترینگ، مؤلفه های نویز به طور محسوسی کاهش یافته اند.
- در بخش رمزگشایی، با استفاده از اطلاعات کاهش دامنه (attenuation) و ضریب نمونه برداری استخراج شده از نام فایل، پیام پنهان را ابتدا از فایل بدون نویز و سپس (پس از فیلترینگ هدفمند برای حذف نویز اعمال شده بر روی فایل صوتی) از فایل نویزی بازیابی کردیم.

نتیجه نهایی: فیلتر تطبیقی (LMS) همراه با فیلترهای کلاسیک FIR/IIR مجموعاً ابزارهایی کارا و مفید برای حذف نویز هستند و همچنین امکان بازگردانی دقیق پیام های مخفی نهفته در یک سیگنال را نیز میتوانند در مواردی برای ما فراهم کنند.